

Labyrinth Breach – Implementation Deep Dive

System Architecture, Code Walkthrough & Live Demo

Ahmed Jawed

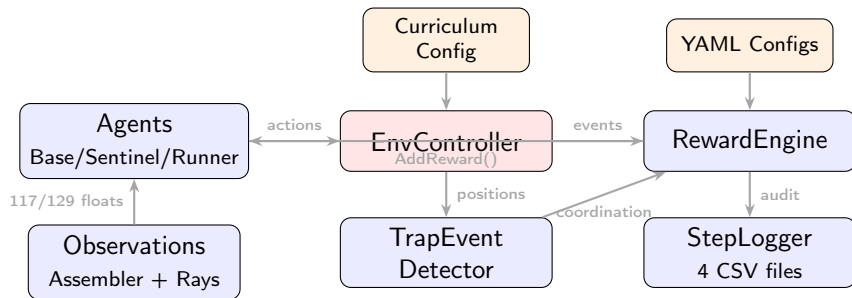
AI641 – AI for Robotics | MS AI, LUMS

May 2025

What I Will Cover

- ➊ **System Architecture** – how the pieces connect
- ➋ **Observation Pipeline** – how agents see the world
- ➌ **Reward Engine** – how we shape behavior
- ➍ **Training Pipeline** – curriculum & config flow
- ➎ **Logging & Evaluation** – how we measure results
- ➏ **Live Demo** – trained agents in the dynamic maze

System Architecture Overview



Key design: Everything is config-driven. Change reward weights, curriculum thresholds, or observation setup without touching C# code.

Agent Hierarchy

Inheritance Chain

- `Unity.MLAgents.Agent`
 - `BaseAgent` – movement, observation collection, ML-Agents hooks
 - `SentinelAgent` – capture radius, target tracking
 - `RunnerAgent` – escape logic, captured state

Parameter Sharing

- All 3 Sentinels → one PPO brain
- Both Runners → one PPO brain
- Same weights, different observations
- What one agent learns, all agents know

ML-Agents Lifecycle (every step)

- 1 **Observe** – `CollectObservations` builds the 117/129 float vector from sensors, rays, and memory
- 2 **Decide** – neural network maps observation → 2 continuous actions (`moveX`, `moveZ`)
- 3 **Act** – `OnActionReceived` applies movement to the agent's rigidbody
- 4 **Reward** – `RewardEngine` computes and assigns reward
- 5 **Log** – `StepLogger` records the event to CSV

Episode boundaries

- Start: agents reset position and memory

Observation Pipeline – How Agents See

Component	Sent.	Run.
Self state	10	10
Env context	7	7
Rays ($N \times 6$)	84	96
LOS memory	6	6
Opponent summary	10	10
Total	117	129

Each ray = 6 floats:

- 1 Normalized hit distance
- 2 Wall (one-hot)
- 3 Exit zone (one-hot)
- 4 Teammate (one-hot)
- 5 Opponent (one-hot)

Design Decisions

- **No camera/pixels** – vector observations train 10x faster and are fully interpretable
- **No global state** – agents never see the full maze, making this a true POSG
- **Memory** – stores last-known XYZ when target leaves LOS, decays over time
- **Opponent summary** – 2 nearest enemies: distance, relative position, visibility, threat pressure

Why more Runner rays?

Runners are prey – they need wider 360^{5/10}

Reward Engine – How We Shape Behavior

Sentinel Rewards

Event	Weight
Team capture (win)	+1.0
Timeout/escape (loss)	-1.0
Individual capture	+0.1
Chase progress	+0.003
Chase regression	-0.0025
Wall loop penalty	-0.002
Exploration bonus	+0.0008

Runner Rewards

Event	Weight
Team exit (win)	+1.0
Both captured (loss)	-1.0
Survival/step	+0.0005
Evade progress	+0.0025

Reward Pipeline

YAML config → RewardConfig (parse) →
Role-specific policy → RewardEngine (apply) →
agent.AddReward() → StepLogger (CSV audit)

Key design: Terminal rewards are ± 1.0 .
Shaping rewards are ~ 0.001 – 0.003 . This ensures
the agent always prioritizes winning over farming
intermediate bonuses.

Tactical Layer (TrapEventDetector)

- Pincer: 2 Sentinels on opposite sides
- Corridor control: both ends blocked
- Exit denial: guarding exit
- Dead-end forcing
- Cluster penalty (anti-bunching)

Training Pipeline – Curriculum & Config

4-Stage Curriculum

Stage	Ep.	Walls	WR
Static, fixed	1200	–	.62
Static, random	2200	–	.55
Dynamic, 20s	3200	20s	.50
Dynamic, 8s	3000	8s	.45

Advance rule:

Must hit BOTH min episodes AND win rate threshold before advancing.

Why this order?

- 1 Basic pursuit in open space
- 2 Prevent route memorization
- 3 Adapt to slow changes
- 4 React to fast topology shifts

PPO Hyperparameters

Parameter	Value
Learning rate	3×10^{-4}
Batch size	1024
Buffer size	10240
Hidden layers	2×256
Epochs	3
γ	0.99
λ (GAE)	0.95
ϵ (clip)	0.2
β (entropy)	0.005
Time horizon	64
Max steps	500K

Follows MAPPO recipe (Yu 2022)

Logging & Evaluation – How We Measure

4 CSV Output Files

- ① `episode_log.csv`
outcome, duration, captured count
- ② `agent_step_log.csv`
per-agent XYZ every step
- ③ `reward_audit.csv`
every reward event with type & value
- ④ `replay_events.csv`
captures, escapes, tactical events

Evaluation Protocol

- 3 seeds: 42, 101, 202
- Seen layouts (training mazes)
- Unseen layouts (new procedural mazes)
- Metrics averaged across seeds

Results

Metric	Seen	Unseen
Sentinel WR	62.0%	60.3%
1st capture	17.2s	9.7s
2nd capture	37.8s	24.5s
Escape frac	26.0%	26.8%
Timeout surv	12.0%	12.9%

Key insight:

Win rates transfer (1.7pt drop).

Capture timing shifts (7.5s faster).

Policies are topology-sensitive but not layout-memorized.

Safe claim: “Policies transfer but are topology-sensitive.”

Summary – Implementation Highlights

What makes this more than a toy project:

- Modular architecture (agents, rewards, sensors, logging are all separate)
- Config-driven (YAML for everything: rewards, curriculum, PPO, evaluation)
- Full audit trail (4 CSV logs track every reward, every step, every event)
- Reproducible (3 seeds, seen/unseen split, public repo)
- Dynamic environment (walls shift during episodes)

Honest limitations:

- No MA-POCA comparison yet (credit assignment)
- No ablation study (memory on/off, shaping on/off)
- Coordination is observed qualitatively but not quantified with dedicated metrics
- BufferSensor path exists but is untested

Repo: github.com/ahmedjawedaj/labyrinth-breach-marl